

Future Interns

Cyber Security Internship

Task 1

*Vulnerability Assessment Report
for a Live Website*

Prepared By:

Sathya Ankitha Ravirala

CIN: FIT/JUN26/CS9500

Table of Contents

1 Executive Summary

2 Objective

3 Scope

4 Target Website

5 Tools Used

6 Methodology

7 Findings

8 Risk Classification

9 Screenshots

10 Recommendations

11 Conclusion

12 References

1. Executive Summary

This report presents the results of a passive vulnerability assessment conducted on a publicly accessible demonstration website using industry-standard cybersecurity tools. The assessment was performed as part of the **Future Interns Cyber Security Internship – Task 1**.

The primary objective of this assessment was to identify common web application security weaknesses without exploiting or causing any disruption to the target application. The assessment utilized **Nmap** for network reconnaissance and **OWASP ZAP (Zed Attack Proxy)** for passive web application security analysis. Browser Developer Tools were also used to observe website behaviour and HTTP responses.

During the assessment, several security issues were identified, including **SQL Injection indicators, Reflected Cross-Site Scripting (XSS), Missing Anti-CSRF Tokens, Missing Security Headers, Information Disclosure, and other security configuration weaknesses**. These findings were classified according to their potential security impact and accompanied by practical remediation recommendations.

The assessment demonstrates the importance of regular vulnerability assessments in identifying potential security risks before they can be exploited by attackers. Addressing the identified issues will improve the confidentiality, integrity, and overall security posture of the web application.

2. Objective

The objective of this vulnerability assessment is to evaluate the security posture of a live web application through a **passive security assessment** using industry-standard cybersecurity tools. The assessment was conducted without exploiting or modifying the target application, ensuring that no harm or service disruption occurred during the testing process.

The specific objectives of this assessment are:

- To identify potential security vulnerabilities, present in the target web application.
- To perform network reconnaissance using **Nmap** and web application analysis using **OWASP ZAP (Passive Scan)**.
- To classify the identified vulnerabilities based on their severity and potential business impact.
- To provide practical remediation recommendations for improving the overall security of the application.
- To gain hands-on experience in vulnerability assessment, security reporting, and web application security as part of the Future Interns Cyber Security Internship.

This assessment follows ethical cybersecurity practices and focuses solely on identifying security weaknesses through passive observation and analysis.

3. Scope

This vulnerability assessment was conducted to evaluate the security posture of a publicly accessible demonstration web application using **passive security testing techniques**. The assessment was limited to identifying potential security weaknesses without attempting to exploit vulnerabilities or disrupt the normal operation of the website.

The scope of this assessment included the following activities:

- Performing basic network reconnaissance using **Nmap**.
- Conducting a **passive web application scan** using **OWASP ZAP**.
- Reviewing HTTP requests and responses to identify security misconfigurations.
- Identifying common web application vulnerabilities such as:
 - SQL Injection indicators
 - Cross-Site Scripting (XSS)
 - Missing Anti-CSRF Tokens
 - Missing Security Headers
 - Information Disclosure
- Classifying identified findings based on their severity and potential impact.
- Providing recommendations to improve the security posture of the web application.

Out of Scope

The following activities were intentionally **not performed** during this assessment:

- Active exploitation of vulnerabilities.
- Denial-of-Service (DoS) or stress testing.
- Authentication bypass attempts.
- Password cracking or brute-force attacks.
- Modification, deletion, or manipulation of application data.
- Any activity that could negatively impact the availability or integrity of the target application.

This assessment was conducted following responsible and ethical cybersecurity practices for educational purposes as part of the **Future Interns Cyber Security Internship**.

4. Target Website

Website Name: TestFire Demo Application (Altoro Mutual)

Website URL: <http://demo.testfire.net>

Purpose of Assessment:

The target selected for this assessment is **TestFire (Altoro Mutual)**, a deliberately vulnerable web application designed for security training, vulnerability assessment, and penetration testing practice. The website provides a safe and legal environment for identifying common web application security weaknesses without affecting real-world users or production systems.

The assessment was performed using only **passive and non-destructive techniques**, including network reconnaissance and passive web application scanning. No exploitation, modification of data, or denial-of-service activities were performed during the assessment.

5. Tools Used

The following industry-standard tools were used to perform the vulnerability assessment:

Tool	Purpose
Nmap	Performed network reconnaissance to identify open ports, running services, and basic information about the target host.
OWASP ZAP (Zed Attack Proxy)	Conducted a passive web application security assessment to identify potential security issues without exploiting the application.
Browser Developer Tools	Inspected HTTP requests, response headers, cookies, and client-side resources to support the assessment.
Google Chrome	Accessed and interacted with the target website during the assessment process.
Microsoft Word	Prepared and formatted the vulnerability assessment report in a professional layout.

Why These Tools Were Chosen

These tools were selected because they are widely used in the cybersecurity industry for web application security assessments. Together, they provide a combination of network reconnaissance, passive vulnerability detection, HTTP traffic analysis, and professional documentation while ensuring that the assessment remains safe, ethical, and non-destructive.

6. Methodology

The vulnerability assessment was performed using a structured, passive, and non-destructive approach to identify potential security weaknesses in the target web application. The following methodology was followed:

Step 1: Target Selection

- Selected the demo website <http://demo.testfire.net> as the assessment target.
- Verified that the website is intentionally designed for security testing and educational purposes.

Step 2: Network Reconnaissance using Nmap

- Performed an Nmap scan to identify the target's open ports and available network services.
- Collected basic information about the server without attempting any exploitation.
- Saved the scan results and captured a screenshot as evidence.

Step 3: Passive Web Application Assessment using OWASP ZAP

- Configured OWASP ZAP to assess the target website.
- Used the **Passive Scan** feature to analyze HTTP requests and responses without sending malicious payloads or modifying the application.
- Allowed ZAP to crawl the application and automatically identify potential security issues.
- Exported the HTML report and captured screenshots of the scan results and alerts.

Step 4: Manual Verification

- Reviewed the findings generated by OWASP ZAP.
- Verified the identified issues using Browser Developer Tools where applicable.
- Classified the findings based on their potential security impact.

Step 5: Documentation

- Recorded all observations, screenshots, and exported reports.
- Prepared a professional vulnerability assessment report including findings, risk classification, recommendations, and supporting evidence.

Note:

This assessment was conducted using **passive and ethical testing techniques only**. No attempts were made to exploit vulnerabilities, alter data, bypass authentication, or disrupt the availability of the target website.

7. Findings

The OWASP ZAP passive vulnerability assessment identified multiple security issues in the target web application (<http://demo.testfire.net>). The findings were categorized according to their risk level to help prioritize remediation efforts.

7.1 High-Risk Findings

1. Cross-Site Scripting (Reflected)

Risk Level: High

Description:

The application was found to be vulnerable to **Reflected Cross-Site Scripting (XSS)**. User-supplied input was reflected in the application's response without proper validation or output encoding, allowing malicious JavaScript code to execute in a user's browser.

Potential Impact:

- Session hijacking
- Cookie theft
- Website defacement
- Phishing attacks
- Execution of malicious scripts in users' browsers

2. SQL Injection

Risk Level: High

Description:

The assessment detected indicators of a potential **SQL Injection** vulnerability. User input submitted through the application may be processed by backend database queries without adequate validation or parameterization.

Potential Impact:

- Unauthorized database access
- Retrieval of sensitive information
- Modification or deletion of database records
- Authentication bypass
- Complete compromise of application data

7.2 Medium-Risk Findings

3. Absence of Anti-CSRF Tokens

Risk Level: Medium

Description:

Several forms within the application did not implement Anti-CSRF (Cross-Site Request Forgery) tokens, increasing the risk of unauthorized actions being performed on behalf of authenticated users.

4. Content Security Policy (CSP) Header Not Set

Risk Level: Medium

Description:

The server does not implement a **Content Security Policy (CSP)** header. Without CSP, browsers cannot effectively restrict the execution of untrusted scripts, increasing exposure to Cross-Site Scripting attacks.

5. Missing Anti-Clickjacking Header

Risk Level: Medium

Description:

The application does not include the **X-Frame-Options** or equivalent anti-clickjacking protection headers. This may allow attackers to embed the website inside malicious frames and trick users into performing unintended actions.

6. Subresource Integrity (SRI) Attribute Missing

Risk Level: Medium

Description:

Some externally loaded resources do not use **Subresource Integrity (SRI)** attributes. This reduces assurance that external JavaScript resources have not been modified or tampered with.

7.3 Low-Risk Findings

7. Cookie Without SameSite Attribute

Risk Level: Low

Description:

Cookies issued by the application do not specify the **SameSite** attribute, increasing the possibility of Cross-Site Request Forgery attacks under certain circumstances.

8. Cross-Domain JavaScript Source File Inclusion

Risk Level: Low

Description:

The application loads JavaScript resources from external domains. If those external resources are compromised, malicious scripts may be executed within the application.

9. Server Version Information Disclosure

Risk Level: Low

Description:

The HTTP response headers reveal server version information, which could assist attackers in identifying known vulnerabilities affecting the server software.

10. Missing X-Content-Type-Options Header

Risk Level: Low

Description:

The application does not include the **X-Content-Type-Options** security header. This may allow browsers to incorrectly interpret content types, increasing the risk of content-sniffing attacks.

7.4 Informational Findings

The assessment also identified several informational observations that may help improve the application's security posture.

These include:

- Information Disclosure through Suspicious Comments
- Modern Web Application Detection
- Session Management Response Identified

Although these findings do not represent immediate vulnerabilities, they provide useful information for security monitoring and secure application configuration.

Summary of Findings

The OWASP ZAP assessment identified:

- **2 High-Risk** findings
- **4 Medium-Risk** findings
- **4 Low-Risk** findings
- **3 Informational** findings

The most critical issues identified were **Reflected Cross-Site Scripting (XSS)** and **SQL Injection**, both of which can significantly impact the confidentiality, integrity, and availability of the application if left unaddressed. These findings are taken directly from your scan report.

8. Risk Classification

The identified vulnerabilities were classified based on their severity and potential impact on the security of the web application.

S.No	Vulnerability	Risk Level	Potential Impact
1	Cross-Site Scripting (Reflected)	High	Execution of malicious scripts, session hijacking, cookie theft

S.No	Vulnerability	Risk Level	Potential Impact
2	SQL Injection	High	Unauthorized database access, data leakage, authentication bypass
3	Absence of Anti-CSRF Tokens	Medium	Unauthorized actions performed on behalf of authenticated users
4	Content Security Policy (CSP) Header Not Set	Medium	Increased exposure to Cross-Site Scripting attacks
5	Missing Anti-Clickjacking Header	Medium	Clickjacking attacks through malicious framing
6	Subresource Integrity (SRI) Attribute Missing	Medium	Risk of loading modified or compromised external resources
7	Cookie Without SameSite Attribute	Low	Increased risk of Cross-Site Request Forgery attacks
8	Cross-Domain JavaScript Source File Inclusion	Low	External JavaScript resources could introduce security risks
9	Server Version Information Disclosure	Low	Attackers may identify server software and target known vulnerabilities
10	Missing X-Content-Type-Options Header	Low	Browser content-sniffing attacks

S.No	Vulnerability	Risk Level	Potential Impact
11	Information Disclosure Suspicious Comments	Informational	Reveals implementation details that may assist attackers
12	Modern Web Application Detection	Informational	General information about the application's technology
13	Session Management Response Identified	Informational	Indicates session handling mechanisms for security review

Risk Summary

The vulnerability assessment identified a total of **13 unique alert types**:

- **High Risk: 2**
- **Medium Risk: 4**
- **Low Risk: 4**
- **Informational: 3**

The **High-Risk** vulnerabilities—**Reflected Cross-Site Scripting (XSS)** and **SQL Injection**—should be treated as the highest priority because they can significantly impact the confidentiality, integrity, and availability of the application if exploited. Medium-risk findings should be addressed next to strengthen the application's security posture, while low-risk and informational findings should also be reviewed as part of ongoing security hardening.

9. Screenshots

The following screenshots provide visual evidence of the vulnerability assessment conducted using Nmap and OWASP ZAP. These screenshots support the findings presented in this report.

Figure 1: Nmap Scan Results

Caption: Screenshot showing the Nmap scan results, including discovered open ports and running services on the target system.

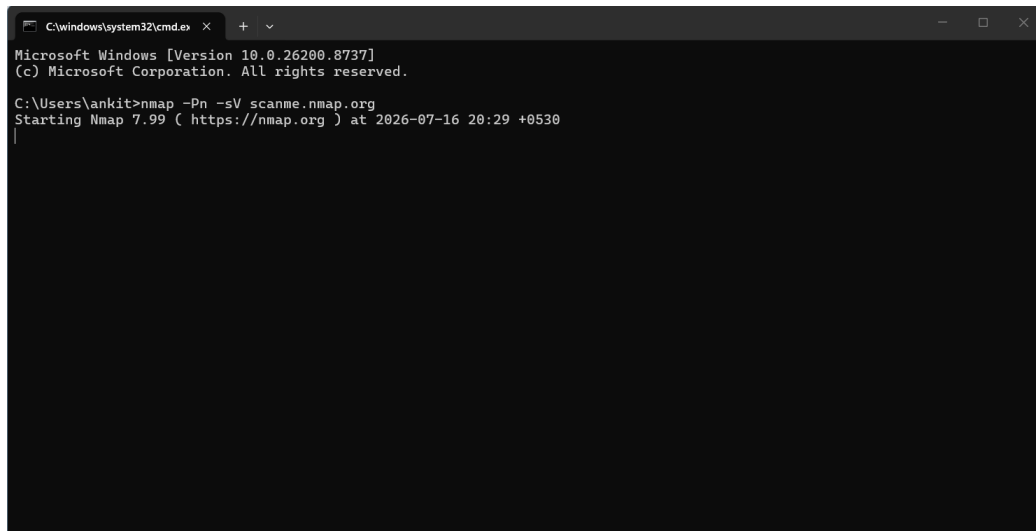


Figure 2: OWASP ZAP Scan Completion

Caption: Screenshot of the OWASP ZAP scan interface after the vulnerability assessment was successfully completed.

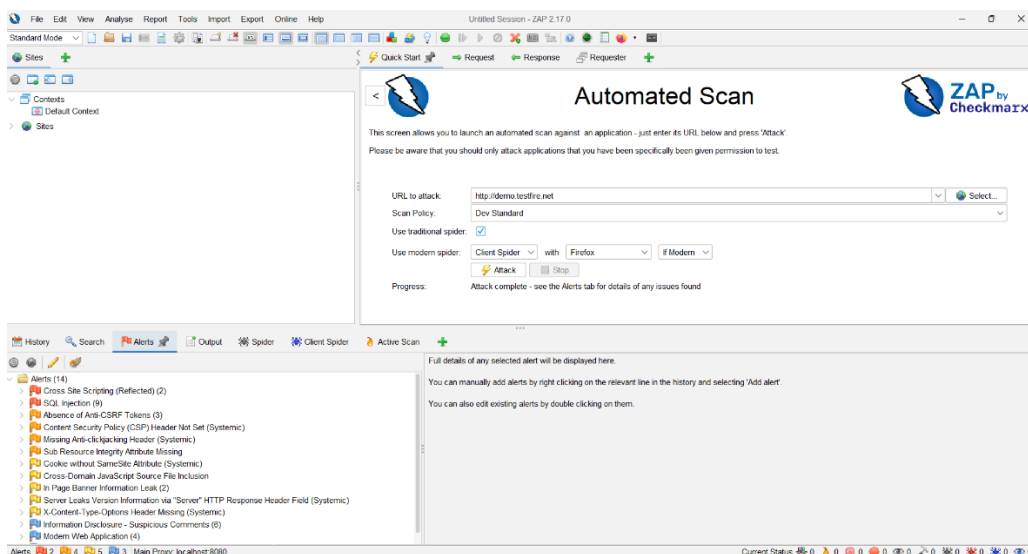


Figure 3: OWASP ZAP Alerts

Caption: Screenshot displaying the identified security alerts and their respective risk levels detected during the assessment.

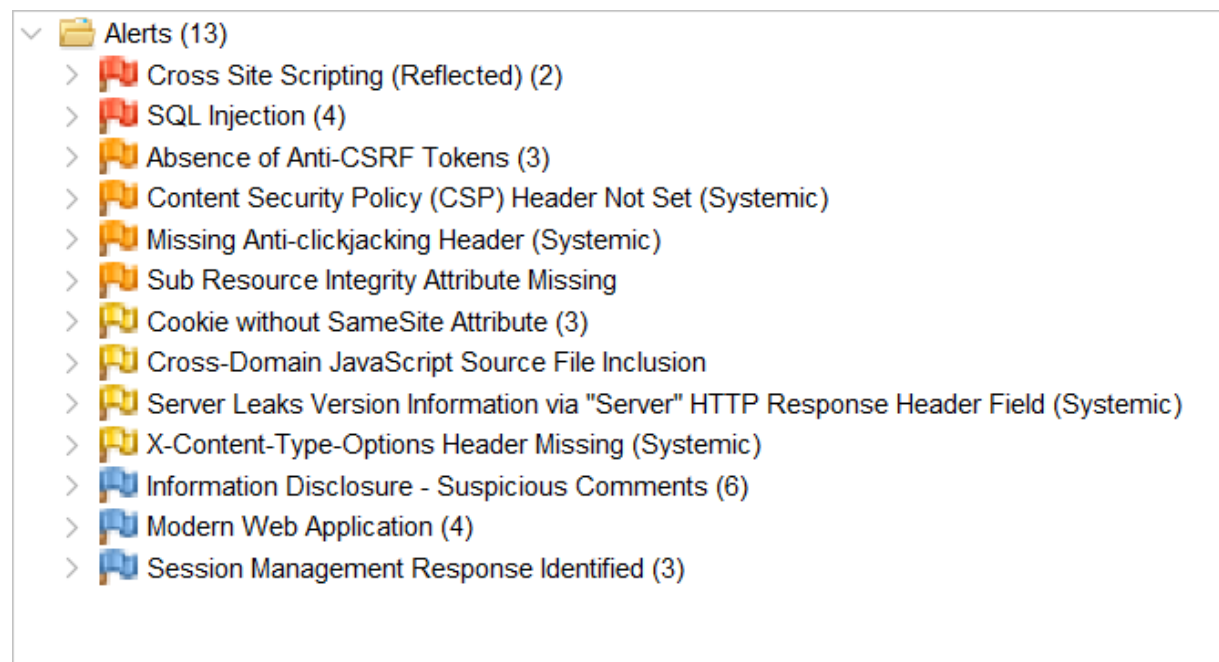
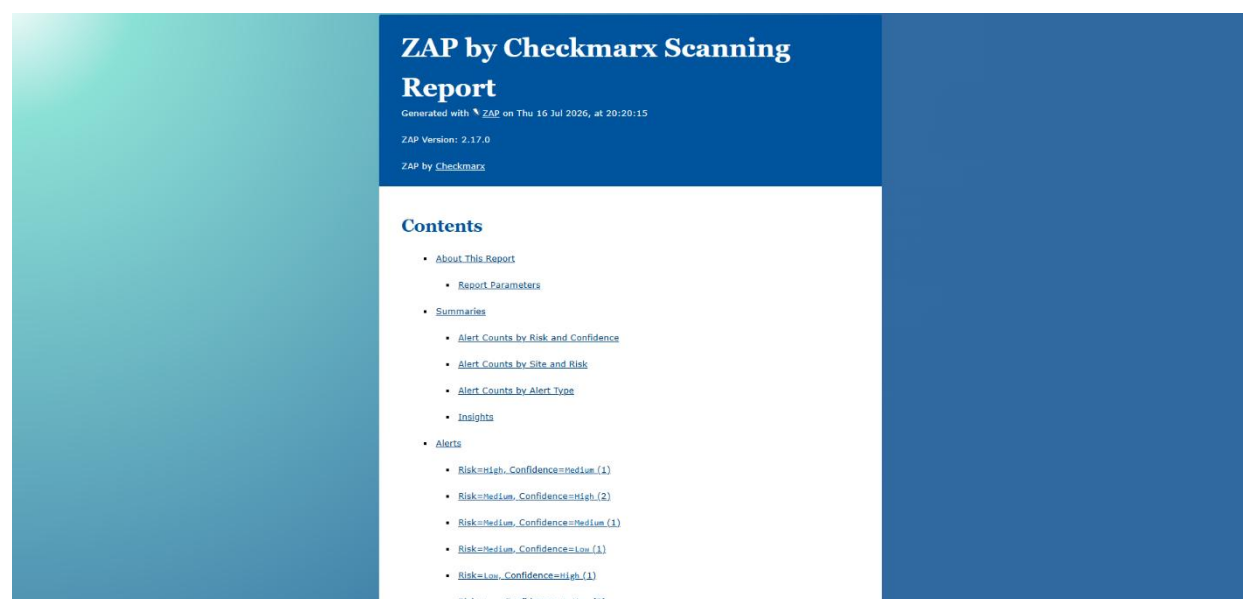


Figure 4: Generated HTML Report

Caption: Screenshot of the generated OWASP ZAP HTML vulnerability assessment report.



10.Recommendations

The following recommendations are provided to mitigate the security issues identified during the vulnerability assessment and improve the overall security posture of the web application.

1. Prevent Cross-Site Scripting (XSS)

- Validate all user inputs on both the client and server sides.
- Encode all user-supplied data before displaying it in web pages.
- Implement a strong Content Security Policy (CSP) to reduce the risk of script injection.
- Use secure web development frameworks that automatically escape untrusted input.

2. Prevent SQL Injection

- Use parameterized queries (Prepared Statements) instead of dynamic SQL queries.
- Validate and sanitize all user inputs.
- Avoid concatenating user input directly into SQL statements.
- Apply the principle of least privilege to database accounts.

3. Protect Against Cross-Site Request Forgery (CSRF)

- Implement Anti-CSRF tokens for all sensitive forms and requests.
- Verify CSRF tokens on the server before processing requests.
- Configure cookies using the **SameSite** attribute where appropriate.

4. Configure Security Headers

Implement recommended HTTP security headers, including:

- Content-Security-Policy (CSP)
- X-Frame-Options

- X-Content-Type-Options
- Referrer-Policy
- Strict-Transport-Security (HSTS)

These headers help protect against common web-based attacks such as clickjacking, content sniffing, and script injection.

5. Secure Cookie Configuration

- Configure cookies with the **HttpOnly**, **Secure**, and **SameSite** attributes.
- Prevent client-side scripts from accessing session cookies whenever possible.

6. Reduce Information Disclosure

- Remove unnecessary comments from HTML source code.
- Hide server version information from HTTP response headers.
- Disable unnecessary debugging information in production environments.

7. Maintain Secure Third-Party Resources

- Use **Subresource Integrity (SRI)** when loading external JavaScript and CSS files.
- Load third-party resources only from trusted sources.
- Regularly update external libraries to their latest secure versions.

8. Perform Regular Security Testing

- Conduct periodic vulnerability assessments using tools such as OWASP ZAP.
- Keep the application and its dependencies updated.
- Perform security testing before every major software release.
- Train developers on secure coding practices and the OWASP Top 10 security risks.

Recommendation Summary

Implementing the above recommendations will significantly reduce the application's exposure to common web security vulnerabilities. Regular vulnerability assessments, secure coding practices, proper input validation, and appropriate security configurations are essential for maintaining a secure web application environment.

11.Conclusion

The vulnerability assessment of the target web application was successfully conducted using **Nmap** for network reconnaissance and **OWASP ZAP** for web application security testing. The assessment identified several security issues, including **Reflected Cross-Site Scripting (XSS)**, **SQL Injection**, missing security headers, the absence of Anti-CSRF protection, cookie configuration weaknesses, and information disclosure findings.

The identified vulnerabilities demonstrate the importance of implementing secure coding practices, proper input validation, secure configuration of HTTP headers, and regular security testing throughout the software development lifecycle. Addressing these issues will significantly improve the application's confidentiality, integrity, and overall security posture.

This assessment was conducted in an ethical manner on the authorized target application solely for educational and security assessment purposes. The findings and recommendations provided in this report can assist developers and security teams in mitigating risks and strengthening the application's defenses against common web-based attacks.

12. References

The following resources and tools were used during the vulnerability assessment and preparation of this report:

1. **OWASP Foundation.** *OWASP Zed Attack Proxy (ZAP).* Available at: <https://www.zaproxy.org/>
2. **OWASP Foundation.** *OWASP Top 10 Web Application Security Risks.* Available at: <https://owasp.org/www-project-top-ten/>
3. **Nmap Project.** *Nmap Network Security Scanner.* Available at: <https://nmap.org/>
4. **TestFire Demo Application.** *Altoro Mutual Demo Website.* Available at: <http://demo.testfire.net/>
5. **Future Interns.** *Cyber Security Internship – Task 1: Vulnerability Assessment Report.* (Internship assignment and guidelines provided by Future Interns.)